EOSC 213: Computational Methods in Geological Engineering

Lecture 14: Stiffness + motivation for implicit methods + Backward Euler and Implicit Midpoint

Shadab Ahamed

Department of Earth, Ocean and Atmospheric Sciences,
The University of British Columbia

Thu, Mar 5, 2026

Logistics

Homework 03

- Will be posted this weekend
- Submission deadline: **20 March, 2026** at 11:59 pm

Logistics

Homework 03

- Will be posted this weekend
- Submission deadline: **20 March, 2026** at 11:59 pm

Midterm II

- Date: **24 March, 2026**
- Will cover material from Lectures 13-18
- Practice Problems Set for Midterm-II (with solutions) will be posted

Plan a head

Start HW03 early and review implicit methods and BVPs in preparation for Midterm II.

Recap: BVPs

Last time:

- Boundary Value Problems (BVPs) specify conditions at multiple points (often at the boundary).

Recap: BVPs

Last time:

- Boundary Value Problems (BVPs) specify conditions at multiple points (often at the boundary).
- Finite differences replace derivatives with algebraic approximations on a grid.

Recap: BVPs

Last time:

- Boundary Value Problems (BVPs) specify conditions at multiple points (often at the boundary).
- Finite differences replace derivatives with algebraic approximations on a grid.
- Discretization typically yields a coupled **linear system**:

$$A\mathbf{T} = \mathbf{b}.$$

Recap: BVPs

Last time:

- Boundary Value Problems (BVPs) specify conditions at multiple points (often at the boundary).
- Finite differences replace derivatives with algebraic approximations on a grid.
- Discretization typically yields a coupled **linear system**:

$$A\mathbf{T} = \mathbf{b}.$$

- We solve for unknowns in $\mathbf{T}$ using numerical linear algebra tools.

Today: Why implicit methods?

In IVPs, explicit methods "march forward":

$$y_{n+1} = y_n + hf(t_n, y_n).$$

Today: Why implicit methods?

In IVPs, explicit methods "march forward":

$$y_{n+1} = y_n + hf(t_n, y_n).$$

But for some problems, explicit methods require **very small** h for stability.

Today: Why implicit methods?

In IVPs, explicit methods "march forward":

$$y_{n+1} = y_n + hf(t_n, y_n).$$

But for some problems, explicit methods require **very small** h for stability.

Today's big idea

Implicit methods require solving for y_{n+1}, but can be **much more stable** (especially for stiff problems).

Roadmap for today

1. Stability viewpoint via the test equation $y' = \lambda y$
2. What is **stiffness**? (intuition + signature)
3. A geoscience stiff example
4. Implicit methods: Backward Euler, Implicit Midpoint
5. Implicit step $\Rightarrow$ solve (linear/nonlinear viewpoint)

Stability refresher: the test equation

Let's study numerical stability using,

$$y'(t) = \lambda y(t), \qquad y(0) = y_0, \qquad \lambda \in \mathbb{C}.$$

Stability refresher: the test equation

Let's study numerical stability using,

$$y'(t) = \lambda y(t), \qquad y(0) = y_0, \qquad \lambda \in \mathbb{C}.$$

Exact solution:

$$y(t) = e^{\lambda t} y_0.$$

Stability refresher: the test equation

Let's study numerical stability using,

$$y'(t) = \lambda y(t), \qquad y(0) = y_0, \qquad \lambda \in \mathbb{C}.$$

Exact solution:

$$y(t) = e^{\lambda t} y_0.$$

If $\Re(\lambda) < 0$, the true solution decays to 0.

Forward Euler: stability condition

Forward Euler:

$$y_{n+1} = y_n + h\lambda y_n = (1 + h\lambda)\, y_n.$$

Forward Euler: stability condition

Forward Euler:

$$y_{n+1} = y_n + h\lambda y_n = (1 + h\lambda)\, y_n.$$

After n steps:

$$y_n = (1 + h\lambda)^n y_0.$$

Forward Euler: stability condition

Forward Euler:

$$y_{n+1} = y_n + h\lambda y_n = (1 + h\lambda)\, y_n.$$

After n steps:

$$y_n = (1 + h\lambda)^n y_0.$$

Stability requirement

To mimic decay when $\Re(\lambda) < 0$, we need

$$|1 + h\lambda| < 1.$$

Forward Euler: stability condition

Forward Euler:

$$y_{n+1} = y_n + h\lambda y_n = (1 + h\lambda)\, y_n.$$

After n steps:

$$y_n = (1 + h\lambda)^n y_0.$$

Stability requirement

To mimic decay when $\Re(\lambda) < 0$, we need

$$|1 + h\lambda| < 1.$$

Implication: if λ is large negative, h must be tiny.

When stability dominates accuracy

Sometimes step size is chosen mainly for **stability**, not accuracy.

When stability dominates accuracy

Sometimes step size is chosen mainly for **stability**, not accuracy.

This motivates implicit methods

If the model contains very fast decaying components (large negative eigenvalues), explicit methods may require extremely small h even when the solution is smooth.

When stability dominates accuracy

Sometimes step size is chosen mainly for **stability**, not accuracy.

This motivates implicit methods

If the model contains very fast decaying components (large negative eigenvalues), explicit methods may require extremely small h even when the solution is smooth.

This phenomenon is called **stiffness**.

What is stiffness?

A problem is often called **stiff** when:

- there are **multiple time scales** (fast decay + slow evolution),

What is stiffness?

A problem is often called **stiff** when:

- there are **multiple time scales** (fast decay + slow evolution),
- explicit methods require **tiny** h for stability,

What is stiffness?

A problem is often called **stiff** when:

- there are **multiple time scales** (fast decay + slow evolution),
- explicit methods require **tiny** h for stability,
- but the solution may not need tiny h for accuracy.

What is stiffness?

A problem is often called **stiff** when:

- there are **multiple time scales** (fast decay + slow evolution),
- explicit methods require **tiny** h for stability,
- but the solution may not need tiny h for accuracy.

In words: "the physics has a fast stable mode that constrains the numerics."

Stiffness signature (linear systems)

Consider a linear system: $\mathbf{y}'(t) = A\mathbf{y}(t)$ with

$$A = \begin{bmatrix} -1 & 0 \\ 0 & -1000 \end{bmatrix}$$

Stiffness signature (linear systems)

Consider a linear system: $\mathbf{y}'(t) = A\mathbf{y}(t)$ with

$$A = \begin{bmatrix} -1 & 0 \\ 0 & -1000 \end{bmatrix}$$

A has eigenvalues with very different orders of magnitude,

$$\lambda_1 = -1, \qquad \lambda_2 = -1000,$$

then the mode with eigenvalue $\lambda_2 = -1000$ decays extremely fast.

Stiffness signature (linear systems)

Consider a linear system: $\mathbf{y}'(t) = A\mathbf{y}(t)$ with

$$A = \begin{bmatrix} -1 & 0 \\ 0 & -1000 \end{bmatrix}$$

A has eigenvalues with very different orders of magnitude,

$$\lambda_1 = -1, \qquad \lambda_2 = -1000,$$

then the mode with eigenvalue $\lambda_2 = -1000$ decays extremely fast.

Stiffness pain point

Forward Euler must choose h small enough for the *fast* mode, even if we only care about the *slow* behavior.

A geoscience example: fast and slow groundwater response

Consider groundwater storage responding to rainfall. Let,

- $S(t)$ = regional groundwater storage (slow evolution)
  - deep aquifer or basin-scale storage

A geoscience example: fast and slow groundwater response

Consider groundwater storage responding to rainfall. Let,

- $S(t)$ = regional groundwater storage (slow evolution)
  - deep aquifer or basin-scale storage
- $R(t)$ = rapid local recharge adjustment (fast relaxation)
  - shallow soil moisture or surface runoff

A geoscience example: fast and slow groundwater response

Consider groundwater storage responding to rainfall. Let,

- $S(t)$ = regional groundwater storage (slow evolution)
  - deep aquifer or basin-scale storage
- $R(t)$ = rapid local recharge adjustment (fast relaxation)
  - shallow soil moisture or surface runoff

A simplified coupled model,

$$S'(t) = -0.1\,S + R,$$
$$R'(t) = -1000R.$$

A geoscience example: fast and slow groundwater response

Consider groundwater storage responding to rainfall. Let,

- $S(t)$ = regional groundwater storage (slow evolution)
  - deep aquifer or basin-scale storage
- $R(t)$ = rapid local recharge adjustment (fast relaxation)
  - shallow soil moisture or surface runoff

A simplified coupled model,

$$S'(t) = -0.1\,S + R,$$
$$R'(t) = -1000R.$$

Interpretation:

- R represents shallow water that drains or equilibrates within hours/days

A geoscience example: fast and slow groundwater response

Consider groundwater storage responding to rainfall. Let,

- $S(t)$ = regional groundwater storage (slow evolution)
  - deep aquifer or basin-scale storage
- $R(t)$ = rapid local recharge adjustment (fast relaxation)
  - shallow soil moisture or surface runoff

A simplified coupled model,

$$S'(t) = -0.1\,S + R,$$
$$R'(t) = -1000R.$$

Interpretation:

- R represents shallow water that drains or equilibrates within hours/days
- S represents deep storage that changes over weeks/months

A geoscience example: fast and slow groundwater response

Consider groundwater storage responding to rainfall. Let,

- $S(t)$ = regional groundwater storage (slow evolution)
  - deep aquifer or basin-scale storage
- $R(t)$ = rapid local recharge adjustment (fast relaxation)
  - shallow soil moisture or surface runoff

A simplified coupled model,

$$S'(t) = -0.1\,S + R,$$
$$R'(t) = -1000R.$$

Interpretation:

- R represents shallow water that drains or equilibrates within hours/days
- S represents deep storage that changes over weeks/months

Two time scales appear naturally

Small, fast reservoir + large, slow reservoir $\Rightarrow$ stiffness.

Why this system is stiff

Writing in the matrix form,

$$\mathbf{y}' = \begin{bmatrix} -0.1 & 1 \\ 0 & -1000 \end{bmatrix} \mathbf{y}, \qquad \text{where } \mathbf{y} = \begin{bmatrix} S \\ R \end{bmatrix}$$

Why this system is stiff

Writing in the matrix form,

$$\mathbf{y}' = \begin{bmatrix} -0.1 & 1 \\ 0 & -1000 \end{bmatrix} \mathbf{y}, \qquad \text{where } \mathbf{y} = \begin{bmatrix} S \\ R \end{bmatrix}$$

Eigenvalues:

$$\lambda_1 = -0.1, \qquad \lambda_2 = -1000.$$

Why this system is stiff

Writing in the matrix form,

$$\mathbf{y}' = \begin{bmatrix} -0.1 & 1 \\ 0 & -1000 \end{bmatrix} \mathbf{y}, \qquad \text{where } \mathbf{y} = \begin{bmatrix} S \\ R \end{bmatrix}$$

Eigenvalues:

$$\lambda_1 = -0.1, \qquad \lambda_2 = -1000.$$

Two very different time scales:

- slow mode time scale, $1/|\lambda_1| \sim 10$
- fast mode time scale, $1/|\lambda_2| \sim 0.001$

Why this system is stiff

Writing in the matrix form,

$$\mathbf{y}' = \begin{bmatrix} -0.1 & 1 \\ 0 & -1000 \end{bmatrix} \mathbf{y}, \qquad \text{where } \mathbf{y} = \begin{bmatrix} S \\ R \end{bmatrix}$$

Eigenvalues:

$$\lambda_1 = -0.1, \qquad \lambda_2 = -1000.$$

Two very different time scales:

- slow mode time scale, $1/|\lambda_1| \sim 10$
- fast mode time scale, $1/|\lambda_2| \sim 0.001$

Forward Euler stability requires roughly: $h \lesssim \frac{2}{|\lambda_2|} = \frac{2}{1000} = 0.002.$

Why this system is stiff

Writing in the matrix form,

$$\mathbf{y}' = \begin{bmatrix} -0.1 & 1 \\ 0 & -1000 \end{bmatrix} \mathbf{y}, \qquad \text{where } \mathbf{y} = \begin{bmatrix} S \\ R \end{bmatrix}$$

Eigenvalues:

$$\lambda_1 = -0.1, \qquad \lambda_2 = -1000.$$

Two very different time scales:

- slow mode time scale, $1/|\lambda_1| \sim 10$
- fast mode time scale, $1/|\lambda_2| \sim 0.001$

Forward Euler stability requires roughly: $h \lesssim \frac{2}{|\lambda_2|} = \frac{2}{1000} = 0.002.$

Key point

We are forced to take tiny steps because of the fast mode, even if we only care about long-term groundwater evolution.

Backward Euler (an implicit method)

We have done Backward Euler before (just mentioning here for completeness).

Backward Euler:

$$y_{n+1} = y_n + hf(t_{n+1}, y_{n+1}).$$

Backward Euler (an implicit method)

We have done Backward Euler before (just mentioning here for completeness).

Backward Euler:

$$y_{n+1} = y_n + hf(t_{n+1}, y_{n+1}).$$

Observation: the unknown y_{n+1} appears on *both sides*.
So each step requires a **solve**.

Backward Euler on the test equation

For $y' = \lambda y$, Backward Euler gives:

$$y_{n+1} = y_n + h\lambda y_{n+1}.$$

Backward Euler on the test equation

For $y' = \lambda y$, Backward Euler gives:

$$y_{n+1} = y_n + h\lambda y_{n+1}.$$

Bring terms together:

$$(1 - h\lambda)\, y_{n+1} = y_n \quad \Rightarrow \quad y_{n+1} = \frac{1}{1 - h\lambda}\, y_n.$$

Backward Euler on the test equation

For $y' = \lambda y$, Backward Euler gives:

$$y_{n+1} = y_n + h\lambda y_{n+1}.$$

Bring terms together:

$$(1 - h\lambda)\, y_{n+1} = y_n \quad \Rightarrow \quad y_{n+1} = \frac{1}{1 - h\lambda}\, y_n.$$

Amplification factor:

$$R(z) = \frac{1}{1 - z}, \qquad z = h\lambda.$$

Why implicit methods can be more stable

Forward Euler: $R(\lambda h) = 1 + h\lambda \implies R(z) = 1 + z$

Why implicit methods can be more stable

Forward Euler: $R(\lambda h) = 1 + h\lambda \implies R(z) = 1 + z$

Backward Euler: $R(\lambda h) = \frac{1}{1-\lambda h} \implies R(z) = \dfrac{1}{1-z}$

Why implicit methods can be more stable

Forward Euler: $R(\lambda h) = 1 + h\lambda \implies R(z) = 1 + z$

Backward Euler: $R(\lambda h) = \frac{1}{1-\lambda h} \implies R(z) = \dfrac{1}{1-z}$

For stiff decay (λ large negative), Backward Euler remains stable for much larger h.

Why implicit methods can be more stable

Forward Euler: $R(\lambda h) = 1 + h\lambda \implies R(z) = 1 + z$

Backward Euler: $R(\lambda h) = \frac{1}{1-\lambda h} \implies R(z) = \dfrac{1}{1-z}$

For stiff decay (λ large negative), Backward Euler remains stable for much larger h.

Moral

Implicit methods trade **solve per step** for **better stability**.

Implicit Midpoint method

Implicit Midpoint:

$$y_{n+1} = y_n + h\, f\left(t_n + \frac{h}{2}, \frac{y_n + y_{n+1}}{2}\right).$$

Implicit Midpoint method

Implicit Midpoint:

$$y_{n+1} = y_n + h\,f\left(t_n + \frac{h}{2}, \frac{y_n + y_{n+1}}{2}\right).$$

Why it is interesting:

- Second-order accurate (often more accurate than Backward Euler)

Implicit Midpoint method

Implicit Midpoint:

$$y_{n+1} = y_n + h\,f\left(t_n + \frac{h}{2}, \frac{y_n + y_{n+1}}{2}\right).$$

Why it is interesting:

- Second-order accurate (often more accurate than Backward Euler)
- Still implicit (still requires solving for y_{n+1})

Explicit vs Implicit Midpoint methods

Explicit Midpoint (known)

$$y_{n+\frac{1}{2}} = y_n + \frac{h}{2} f(t_n, y_n)$$

$$y_{n+1} = y_n + h\, f\left(t_n + \frac{h}{2}, y_{n+\frac{1}{2}}\right)$$

Characteristics

- Explicit update
- No equation solve
- Second-order accurate
- Stability still limited

Explicit vs Implicit Midpoint methods

Explicit Midpoint (known)

$$y_{n+\frac{1}{2}} = y_n + \frac{h}{2} f(t_n, y_n)$$

$$y_{n+1} = y_n + h\, f\left(t_n + \frac{h}{2}, y_{n+\frac{1}{2}}\right)$$

Implicit Midpoint (new)

$$y_{n+1} = y_n + h\, f\left(t_n + \frac{h}{2}, \frac{y_n + y_{n+1}}{2}\right)$$

Characteristics

- Explicit update
- No equation solve
- Second-order accurate
- Stability still limited

Explicit vs Implicit Midpoint methods

Explicit Midpoint (known)

$$y_{n+\frac{1}{2}} = y_n + \frac{h}{2} f(t_n, y_n)$$

$$y_{n+1} = y_n + h\, f\left(t_n + \frac{h}{2}, y_{n+\frac{1}{2}}\right)$$

Characteristics

- Explicit update
- No equation solve
- Second-order accurate
- Stability still limited

Implicit Midpoint (new)

$$y_{n+1} = y_n + h\, f\left(t_n + \frac{h}{2}, \frac{y_n + y_{n+1}}{2}\right)$$

Characteristics

- Implicit update
- Requires solving for y_{n+1}
- Second-order accurate
- Much better stability

Explicit vs Implicit Midpoint methods

Explicit Midpoint (known)

$$y_{n+\frac{1}{2}} = y_n + \frac{h}{2} f(t_n, y_n)$$

$$y_{n+1} = y_n + h\, f\left(t_n + \frac{h}{2}, y_{n+\frac{1}{2}}\right)$$

Characteristics

- Explicit update
- No equation solve
- Second-order accurate
- Stability still limited

Implicit Midpoint (new)

$$y_{n+1} = y_n + h\, f\left(t_n + \frac{h}{2}, \frac{y_n + y_{n+1}}{2}\right)$$

Characteristics

- Implicit update
- Requires solving for y_{n+1}
- Second-order accurate
- Much better stability

Same midpoint idea — explicit uses predicted midpoint, implicit uses true midpoint.

Midpoint methods on the test equation $y' = \lambda y$

Explicit Midpoint

$$y_{n+1} = \left(1 + z + \frac{z^2}{2}\right) y_n, \quad z = h\lambda$$

Midpoint methods on the test equation $y' = \lambda y$

Explicit Midpoint

$$y_{n+1} = \left(1 + z + \frac{z^2}{2}\right) y_n, \quad z = h\lambda$$

Amplification factor:

$$R(z) = 1 + z + \frac{z^2}{2}.$$

Midpoint methods on the test equation $y' = \lambda y$

Explicit Midpoint

$$y_{n+1} = \left(1 + z + \frac{z^2}{2}\right) y_n, \quad z = h\lambda$$

Amplification factor:

$$R(z) = 1 + z + \frac{z^2}{2}.$$

- Polynomial growth
- Stability region limited

Implicit Midpoint

$$y_{n+1} = \frac{1 + z/2}{1 - z/2} y_n$$

Midpoint methods on the test equation $y' = \lambda y$

Explicit Midpoint

$$y_{n+1} = \left(1 + z + \frac{z^2}{2}\right) y_n, \quad z = h\lambda$$

Amplification factor:

$$R(z) = 1 + z + \frac{z^2}{2}.$$

- Polynomial growth
- Stability region limited

Implicit Midpoint

$$y_{n+1} = \frac{1 + z/2}{1 - z/2} y_n$$

Amplification factor:

$$R(z) = \frac{1 + z/2}{1 - z/2}.$$

Midpoint methods on the test equation $y' = \lambda y$

Explicit Midpoint

$$y_{n+1} = \left(1 + z + \frac{z^2}{2}\right) y_n, \quad z = h\lambda$$

Amplification factor:

$$R(z) = 1 + z + \frac{z^2}{2}.$$

- Polynomial growth
- Stability region limited

Implicit Midpoint

$$y_{n+1} = \frac{1 + z/2}{1 - z/2} y_n$$

Amplification factor:

$$R(z) = \frac{1 + z/2}{1 - z/2}.$$

- Rational function
- Much larger stability region

Midpoint methods on the test equation $y' = \lambda y$

Explicit Midpoint

$$y_{n+1} = \left(1 + z + \frac{z^2}{2}\right) y_n, \quad z = h\lambda$$

Amplification factor:

$$R(z) = 1 + z + \frac{z^2}{2}.$$

- Polynomial growth
- Stability region limited

Implicit Midpoint

$$y_{n+1} = \frac{1 + z/2}{1 - z/2} y_n$$

Amplification factor:

$$R(z) = \frac{1 + z/2}{1 - z/2}.$$

- Rational function
- Much larger stability region

Implicit methods stabilize stiff decay modes.

Implicit step as an equation to solve

Backward Euler step:

$$y_{n+1} = y_n + hf(t_{n+1}, y_{n+1})$$

Implicit step as an equation to solve

Backward Euler step:

$$y_{n+1} = y_n + hf(t_{n+1}, y_{n+1})$$

Rearrange into a root-finding form:

$$g(y_{n+1}) := y_{n+1} - y_n - hf(t_{n+1}, y_{n+1}) = 0.$$

Implicit step as an equation to solve

Backward Euler step:

$$y_{n+1} = y_n + hf(t_{n+1}, y_{n+1})$$

Rearrange into a root-finding form:

$$g(y_{n+1}) := y_{n+1} - y_n - hf(t_{n+1}, y_{n+1}) = 0.$$

- If f is linear in y, this becomes a **linear system**.

Implicit step as an equation to solve

Backward Euler step:

$$y_{n+1} = y_n + hf(t_{n+1}, y_{n+1})$$

Rearrange into a root-finding form:

$$g(y_{n+1}) := y_{n+1} - y_n - hf(t_{n+1}, y_{n+1}) = 0.$$

- If f is linear in y, this becomes a **linear system**.
- If f is nonlinear, this becomes a **nonlinear equation** (solve iteratively).

Implicit step as an equation to solve

Backward Euler step:

$$y_{n+1} = y_n + hf(t_{n+1}, y_{n+1})$$

Rearrange into a root-finding form:

$$g(y_{n+1}) := y_{n+1} - y_n - hf(t_{n+1}, y_{n+1}) = 0.$$

- If f is linear in y, this becomes a **linear system**.
- If f is nonlinear, this becomes a **nonlinear equation** (solve iteratively).

(We will formalize Newton iteration method in the future.)

Backward Euler for a linear ODE system

Now consider a linear system,

$$\mathbf{y}' = A\mathbf{y},$$

where A is a matrix.

Backward Euler for a linear ODE system

Now consider a linear system,

$$\mathbf{y}' = A\mathbf{y},$$

where A is a matrix.

Backward Euler gives,

$$\mathbf{y}_{n+1} = \mathbf{y}_n + hA\mathbf{y}_{n+1}.$$

Backward Euler for a linear ODE system

Now consider a linear system,

$$\mathbf{y}' = A\mathbf{y},$$

where A is a matrix.

Backward Euler gives,

$$\mathbf{y}_{n+1} = \mathbf{y}_n + hA\mathbf{y}_{n+1}.$$

Bring unknown terms to one side:

$$\mathbf{y}_{n+1} - hA\mathbf{y}_{n+1} = \mathbf{y}_n.$$

Backward Euler for a linear ODE system

Now consider a linear system,

$$\mathbf{y}' = A\mathbf{y},$$

where A is a matrix.

Backward Euler gives,

$$\mathbf{y}_{n+1} = \mathbf{y}_n + hA\mathbf{y}_{n+1}.$$

Bring unknown terms to one side:

$$\mathbf{y}_{n+1} - hA\mathbf{y}_{n+1} = \mathbf{y}_n.$$

Factor:

$$(I - hA)\mathbf{y}_{n+1} = \mathbf{y}_n.$$

Each time step requires solving a linear system

We must solve,

$$(I - hA)\mathbf{y}_{n+1} = \mathbf{y}_n.$$

Each time step requires solving a linear system

We must solve,

$$(I - hA)\mathbf{y}_{n+1} = \mathbf{y}_n.$$

This has the exact same structure as the system arriving in BVP:

$$A\mathbf{T} = \mathbf{b}.$$

- Unknown: $\mathbf{y}_{n+1}$
- Matrix: $I - hA$
- Right-hand side: $\mathbf{y}_n$

Each time step requires solving a linear system

We must solve,

$$(I - hA)\mathbf{y}_{n+1} = \mathbf{y}_n.$$

This has the exact same structure as the system arriving in BVP:

$$A\mathbf{T} = \mathbf{b}.$$

- Unknown: $\mathbf{y}_{n+1}$
- Matrix: $I - hA$
- Right-hand side: $\mathbf{y}_n$

Connection to Lecture 13

BVP: solve once globally in space.
Implicit ODE: solve at every time step.

Summary: Connection between BVP and implicit time stepping

Lecture 13: BVP

Spatial discretization:

$$A\mathbf{T} = \mathbf{b}$$

Summary: Connection between BVP and implicit time stepping

Lecture 13: BVP

Spatial discretization:

$$A\mathbf{T} = \mathbf{b}$$

Solve once to get temperature profile.

Lecture 14: Backward Euler

Time discretization:

$$(I - hA)\mathbf{y}_{n+1} = \mathbf{y}_n$$

Summary: Connection between BVP and implicit time stepping

Lecture 13: BVP

Spatial discretization:

$$A\mathbf{T} = \mathbf{b}$$

Solve once to get temperature profile.

Lecture 14: Backward Euler

Time discretization:

$$(I - hA)\mathbf{y}_{n+1} = \mathbf{y}_n$$

Solve at each time step.

Summary: Connection between BVP and implicit time stepping

Lecture 13: BVP
Spatial discretization:

$$A\mathbf{T} = \mathbf{b}$$

Solve once to get temperature profile.

Lecture 14: Backward Euler
Time discretization:

$$(I - hA)\mathbf{y}_{n+1} = \mathbf{y}_n$$

Solve at each time step.

Implicit time stepping turns ODE solving into repeated linear solves.

Computational viewpoint: BVP vs implicit time stepping

Lecture 13: BVP (space)

Discretization gives:

$$A\mathbf{T} = \mathbf{b}.$$

```
import torch
# solve A T = b
T = torch.linalg.solve(A, b)
```

Computational viewpoint: BVP vs implicit time stepping

Lecture 13: BVP (space)

Discretization gives:

$$A\mathbf{T} = \mathbf{b}.$$

```
import torch
# solve A T = b
T = torch.linalg.solve(A, b)
```

Solve once to get the spatial profile.

Lecture 14: Implicit step (time)

Backward Euler (linear case):

$$(I - hA)\mathbf{y}_{n+1} = \mathbf{y}_n.$$

```
import torch

I = torch.eye(A.shape[0])
M = I - h*A
# solve M y[n+1] = y[n] for each n
for n in range(0, N+1):
      y[n+1] = torch.linalg.solve(M, y[n])
```

Computational viewpoint: BVP vs implicit time stepping

Lecture 13: BVP (space)

Discretization gives:

$$A\mathbf{T} = \mathbf{b}.$$

```
import torch
# solve A T = b
T = torch.linalg.solve(A, b)
```

Solve once to get the spatial profile.

Lecture 14: Implicit step (time)

Backward Euler (linear case):

$$(I - hA)\mathbf{y}_{n+1} = \mathbf{y}_n.$$

```
import torch

I = torch.eye(A.shape[0])
M = I - h*A
# solve M y[n+1] = y[n] for each n
for n in range(0, N+1):
      y[n+1] = torch.linalg.solve(M, y[n])
```

Solve at every time step.

Computational viewpoint: BVP vs implicit time stepping

Lecture 13: BVP (space)

Discretization gives:

$$A\mathbf{T} = \mathbf{b}.$$

```
import torch
# solve A T = b
T = torch.linalg.solve(A, b)
```

Solve once to get the spatial profile.

Lecture 14: Implicit step (time)

Backward Euler (linear case):

$$(I - hA)\mathbf{y}_{n+1} = \mathbf{y}_n.$$

```
import torch

I = torch.eye(A.shape[0])
M = I - h*A
# solve M y[n+1] = y[n] for each n
for n in range(0, N+1):
      y[n+1] = torch.linalg.solve(M, y[n])
```

Solve at every time step.

Same tool: linear algebra solver

Different use: once (space) vs repeated (time)

Summary for today

- Stability can force h to be small (even when accuracy does not).

Summary for today

- Stability can force h to be small (even when accuracy does not).
- **Stiffness:** multiple time scales $\Rightarrow$ explicit methods become inefficient.

Summary for today

- Stability can force h to be small (even when accuracy does not).
- **Stiffness:** multiple time scales $\Rightarrow$ explicit methods become inefficient.
- Geoscience example: fast recharge adjustment + slow storage evolution $\Rightarrow$ stiffness.

Summary for today

- Stability can force h to be small (even when accuracy does not).
- **Stiffness:** multiple time scales $\Rightarrow$ explicit methods become inefficient.
- Geoscience example: fast recharge adjustment + slow storage evolution $\Rightarrow$ stiffness.
- **Backward Euler:** implicit, often much more stable for stiff decay problems.

Summary for today

- Stability can force h to be small (even when accuracy does not).
- **Stiffness:** multiple time scales $\Rightarrow$ explicit methods become inefficient.
- Geoscience example: fast recharge adjustment + slow storage evolution $\Rightarrow$ stiffness.
- **Backward Euler:** implicit, often much more stable for stiff decay problems.
- **Implicit midpoint:** implicit + second-order accurate; good stability behavior.

Summary for today

- Stability can force h to be small (even when accuracy does not).
- **Stiffness:** multiple time scales $\Rightarrow$ explicit methods become inefficient.
- Geoscience example: fast recharge adjustment + slow storage evolution $\Rightarrow$ stiffness.
- **Backward Euler:** implicit, often much more stable for stiff decay problems.
- **Implicit midpoint:** implicit + second-order accurate; good stability behavior.
- Implicit methods connect naturally to Lecture 13's "solve globally" viewpoint.

What's next

Next lectures:

- Nonlinear systems and numerical issues
- Newton iteration (how we solve $g(y_{n+1}) = 0$ efficiently)
- Review + coding session connecting implicit methods and BVPs